LPP 3CFU – Domande d'esame A.A. 25/26
Parte 3

Luca Roversi

December 3, 2025

CT0380 Applicative functors

Exercise 1. Justify why applicative functors are useful for programmers, using an example. □

CT0390 Applicative functors in Haskell

Exercise 2. Recall the definition of the data-type "`Maybe a`". Assume that you already proved that it is a functor.

1. Define the functions `pure` and `(<*>)`.
2. Prove that `(<*>)` enjoys the applicative composition law. □

Exercise 3. Recall the definition of the data-type "`Either a b`". Assume that you already proved that it is a functor.

1. Define the functions `pure` and `(<*>)`.
2. Prove that `(<*>)` enjoys the applicative composition law. □

Exercise 4. Define a data-type "`List a`" that recursively formalizes the corresponding concept.

1. Show that it can be a functor by defining the function `fmap`.
2. Define the functions `pure` and `(<*>)`.
3. Recall the corresponding applicative composition law, explicitly writing the types its components. □

Exercise 5. Consider the following Haskell data-type:

```
newtype Reader r a where
    Reader :: (r -> a) -> Reader r a
```

1. Show that it can be a functor by defining the function `fmap`.
2. Define the functions `pure` and `(<*>)`.
3. Recall the corresponding applicative composition law, explicitly writing the types its components. □

Exercise 6. Consider the following Haskell data-type:

```
newtype Diagonal a = Diagonal {runDiagonal :: (,) a a}
```

1. Show that it can be a functor by defining the function `fmap`.
2. Define the functions `pure` and `(<*>)`.
3. Recall the corresponding applicative composition, explicitly writing the types of the categoriacl objects involved. □

Exercise 7. Recall the definition of the Haskell data-type:

```
newtype Const c a = Const { getConst :: c }
```

Assume that you already proved that it is a functor (and assume that `c` is a monoid).

1. Define the functions `pure` and `(<*>)`.
2. Prove that `(<*>)` enjoys the applicative composition law. □

Exercise 8. Recall the definition of the Haskell data-type:

```
newtype ZipList a = ZipList { getZipList :: [a] }
```

Assume that you already proved that it is a functor by defining:

```
instance Functor ZipList where
  fmap :: (a -> b) -> ZipList a -> ZipList b
  fmap f zl = ZipList (map f (getZipList zl))
```

1. Define the functions `pure` and `(<*>)` (the latter acting pointwise).
2. Prove that `(<*>)` enjoys the applicative composition law. □

Exercise 9. Recall the definition of the Haskell data-type:

```
newtype Identity a = Identity { runIdentity :: a }
```

Assume that you already proved that it is a functor by defining:

1. Define the functions `pure` and `(<*>)`.
2. Prove that `(<*>)` enjoys the applicative composition law. □